**ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL**

**FACULTAD DE INGENIERÍA EN ELECTRICIDAD Y COMPUTACIÓN**

**DISEÑO DE SOFTWARE**

# TAREA DE PRINCIPIOS SOLID Y UML

## Sistema PetDayCare (Grupo 4)

**Integrantes:**

James Matthew Santana Salcedo

Maria Belen Llano Ibarra

Fecha: 31 de octubre de 2026

## Índice de contenido

Sección A: Casos de uso....3
Asunciones....3
Diagrama de casos de uso....4
Descripción de los casos de uso seleccionados....4
Sección B: Diagrama de clases....7
Justificación de los principios SOLID....11
Sección C: Diagramas de secuencia....12
Sección D: Código base en Java....14

## Sección A: Casos de uso

PetDayCare es un sistema para reservar servicios de cuidado temporal de mascotas (guardería diurna, hospedaje, paseos, entrenamiento y bienestar) en centros afiliados. El cliente registra a su mascota, reserva y paga; el administrador configura los servicios del centro; y los incidentes los atiende el personal del centro, con escalamiento a atención general.

### Asunciones

1. Cada servicio tiene una categoría (guardería diurna, hospedaje, paseos, etc.) y un número de cupos que resume la capacidad del lugar y el personal disponible.
2. Una reserva es de una mascota registrada del cliente, para hoy o una fecha futura, y solo si hay cupo. Una misma reserva no se procesa dos veces. Las opciones adicionales (reportes en tiempo real, cámaras en vivo, atención veterinaria) se agregan como complementos con costo.
3. El cupo se bloquea 15 minutos (el enunciado no fija la duración). Si no se paga en ese tiempo, el cupo se libera y la reserva se cancela.
4. El cliente paga un monto que debe cubrir el total. La conexión con un sistema de pagos real queda fuera del alcance.
5. Cada servicio define los días de anticipación necesarios para reembolso total al cancelar. Solo se reembolsa lo que ya se pagó.
6. Las notificaciones se envían por correo en la plataforma web y por mensajería en la plataforma móvil.
7. Los incidentes los atiende primero el personal del centro; si son complejos, se escalan a atención general.
8. No se modelan horarios, promociones, restricciones por tipo de mascota, paquetes, fechas recurrentes ni autenticación de usuarios.

## Diagrama de casos de uso




*Figura 1. Diagrama de casos de uso de PetDayCare (el actor Usuario es el cliente de las descripciones). Se describen a continuación: Reservar servicio, Pagar y confirmar reserva (Confirmar reserva y completar pago), Reportar incidente y Configurar servicios.*

## Descripción de los casos de uso seleccionados

### Caso de uso 1: Reservar servicio

| Caso de Uso | |
|---|---|
| **Título** | Reservar servicio |
| **Autor** | Grupo 4, PetDayCare |
| **Actor principal** | Cliente |
| **Actor secundario** | Ninguno |
| **Incluye a** | Verificar disponibilidad; Bloquear cupo |
| **Es extendido por** | Elegir cuidador; Agregar servicio complementario |
| **Descripción** | El cliente reserva un servicio para una de sus mascotas en una fecha. El sistema verifica la disponibilidad y bloquea el cupo 15 minutos, hasta que el cliente pague. |
| **Precondición** | 1. El cliente tiene una mascota registrada.<br>2. El centro tiene el servicio configurado. |

| Caso de Uso | |
|---|---|
| **Secuencia normal** | 1. El cliente elige el servicio, la mascota y la fecha.<br>2. [Extiende: Elegir cuidador; Agregar servicio complementario] Si lo desea, elige un cuidador y agrega servicios complementarios.<br>3. El sistema libera los cupos de las reservas cuyo bloqueo venció.<br>4. [Incluye: Verificar disponibilidad] El sistema verifica que haya cupo y que la fecha no haya pasado.<br>5. [Incluye: Bloquear cupo] El sistema bloquea el cupo 15 minutos y calcula el total.<br>6. El sistema guarda la reserva en estado EN_ESPERA.<br>7. El sistema muestra el total a pagar. |
| **Secuencia alternativa** | **A1. No hay cupo o la fecha ya pasó.**<br>A1.1. En el paso 4, el sistema informa el motivo y no guarda la reserva. |
| **Postcondición** | • La reserva queda EN_ESPERA con el cupo bloqueado.<br>• Si no se paga en 15 minutos, el cupo se libera y la reserva se cancela. |

### Caso de uso 2: Pagar y confirmar reserva

| Caso de Uso | |
|---|---|
| **Título** | Pagar y confirmar reserva |
| **Autor** | Grupo 4, PetDayCare |
| **Actor principal** | Cliente |
| **Actor secundario** | Ninguno |
| **Incluye a** | Enviar notificación |
| **Es extendido por** | No aplica |
| **Descripción** | El cliente paga una reserva pendiente. El sistema verifica el pago, confirma la reserva y notifica al cliente. |
| **Precondición** | 1. Existe una reserva del cliente en estado EN_ESPERA.<br>2. El bloqueo de 15 minutos no ha vencido. |
| **Secuencia normal** | 1. El cliente paga la reserva indicando el monto.<br>2. El sistema verifica que el bloqueo no haya vencido y que el monto cubra el total.<br>3. El sistema cambia la reserva a estado OCUPADO.<br>4. [Incluye: Enviar notificación] El sistema notifica la confirmación al cliente. |
| **Secuencia alternativa** | **A1. El bloqueo venció.**<br>A1.1. En el paso 2, el sistema libera el cupo y cancela la reserva.<br>**A2. El monto no cubre el total.**<br>A2.1. En el paso 2, el sistema informa «Pago insuficiente»; la reserva sigue EN_ESPERA. |
| **Postcondición** | • La reserva queda OCUPADO y el cliente recibe la notificación.<br>• Si el pago no es válido, la reserva no se confirma. |

### Caso de uso 3: Reportar incidente

| Caso de Uso | |
|---|---|
| **Título** | Reportar incidente |
| **Autor** | Grupo 4, PetDayCare |
| **Actor principal** | Cliente |
| **Actor secundario** | Personal del centro; Atención general (solo si se escala) |
| **Incluye a** | No aplica |
| **Es extendido por** | Escalar incidente [incidente complejo] |
| **Descripción** | El cliente reporta un incidente de una reserva (cambio de última hora, cancelación o problema con el trato). El personal del centro lo atiende y, si es complejo, se escala. |

| Caso de Uso | |
|---|---|
| **Precondición** | 1. El cliente tiene una reserva registrada. |
| **Secuencia normal** | 1. El cliente describe el problema e indica si es complejo.<br>2. El sistema crea el incidente, asignado al personal del centro.<br>3. El personal del centro lo resuelve.<br>4. El sistema notifica al cliente quién lo atendió. |
| **Secuencia alternativa** | **A1. El incidente es complejo.**<br>A1.1. [Extiende: Escalar incidente] En el paso 3, el sistema lo escala a atención general, que lo resuelve. |
| **Postcondición** | • El incidente queda resuelto y registra qué nivel lo atendió.<br>• El cliente recibe la notificación. |

**Caso de uso 4: Configurar servicio**

| Caso de Uso | |
|---|---|
| **Título** | Configurar servicio |
| **Autor** | Grupo 4, PetDayCare |
| **Actor principal** | Administrador del centro |
| **Actor secundario** | Ninguno |
| **Incluye a** | No aplica |
| **Es extendido por** | No aplica |
| **Descripción** | El administrador agrega un servicio al catálogo de su centro con su categoría, nombre, precio, cupos y días de anticipación para reembolso total. |
| **Precondición** | 1. El administrador está asociado a un centro afiliado. |
| **Secuencia normal** | 1. El administrador ingresa categoría, nombre, precio, cupos y días de reembolso.<br>2. El sistema crea el servicio y lo agrega al catálogo del centro.<br>3. El sistema confirma la configuración. |
| **Secuencia alternativa** | **A1. Datos inválidos.**<br>A1.1. En el paso 1, el sistema pide corregir los datos y no crea el servicio. |
| **Postcondición** | • El servicio queda disponible en el catálogo del centro. |

## Sección B: Diagrama de clases

El diseño aplica dos patrones: fábrica abstracta, con FabricaPlataforma, que crea el notificador y la vista de cada plataforma (móvil y web); y flujo de datos, con Flujo y sus filtros, por los que pasa la reserva en pasos independientes. Las relaciones se muestran con su nombre, multiplicidad y rol. El diagrama completo se presenta en cuatro páginas.

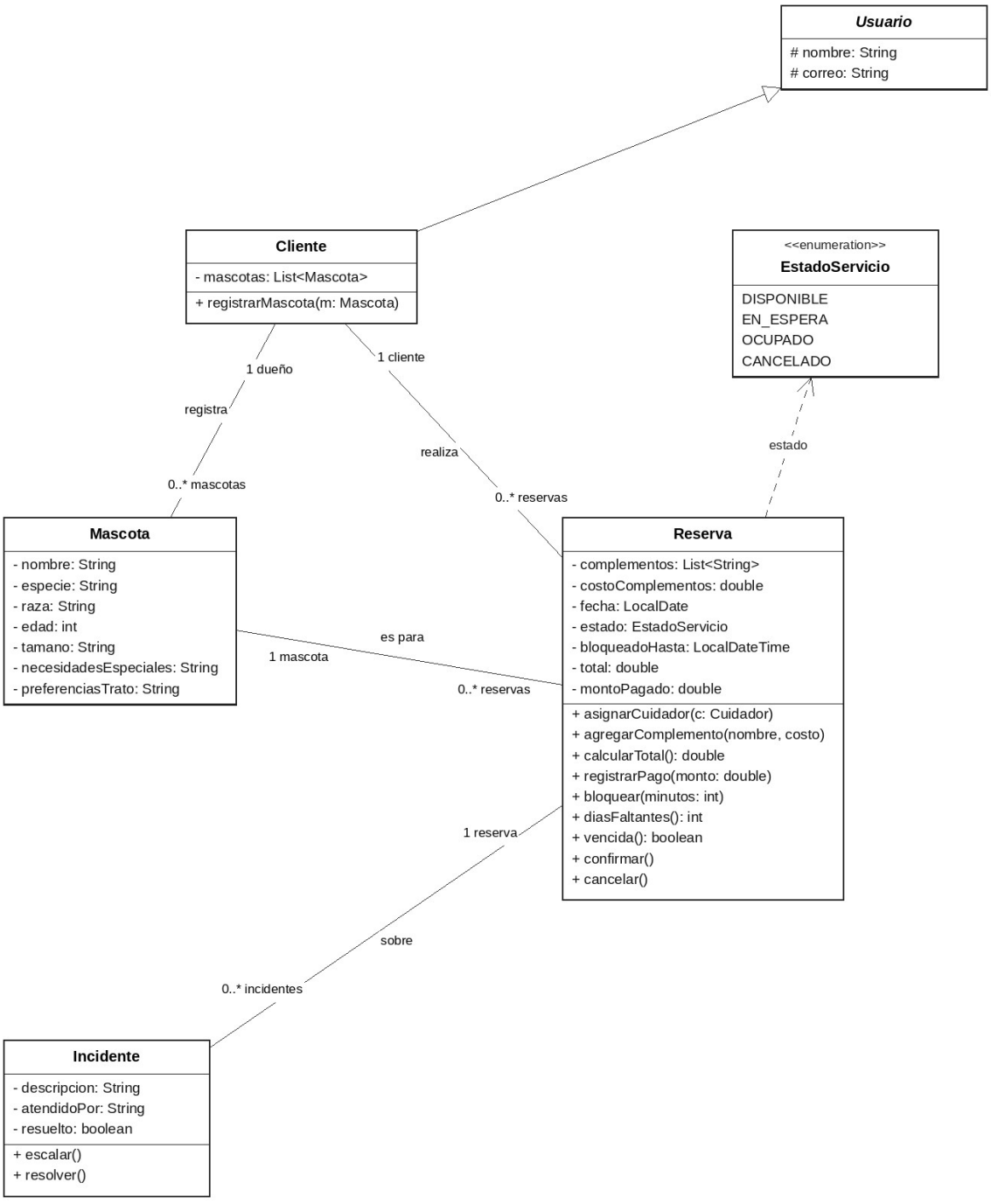



*Figura 2. Diagrama de clases: usuarios, mascotas, reservas e incidentes.*

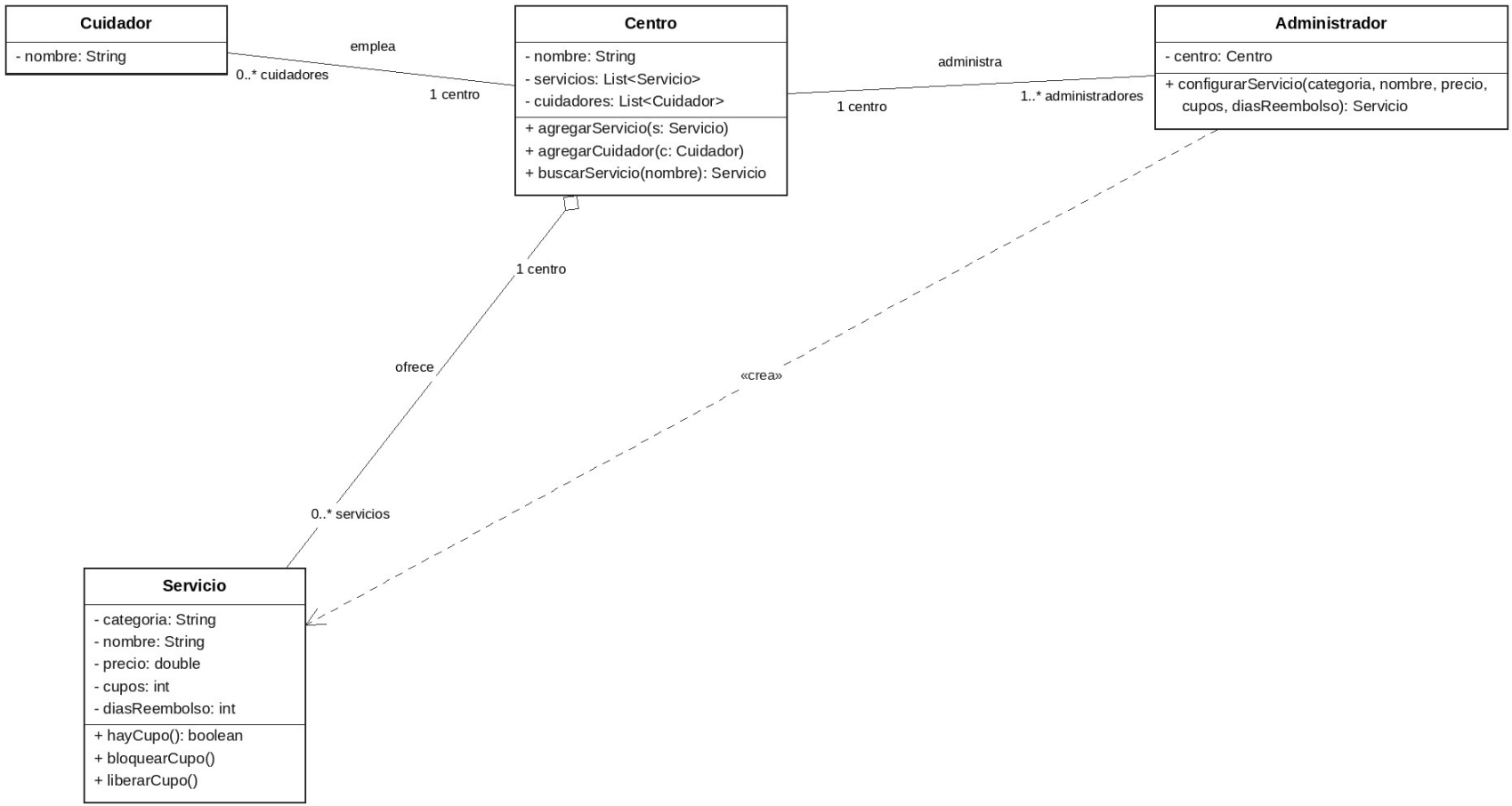



*Figura 3. Diagrama de clases: administrador, centro, cuidadores y servicios.*

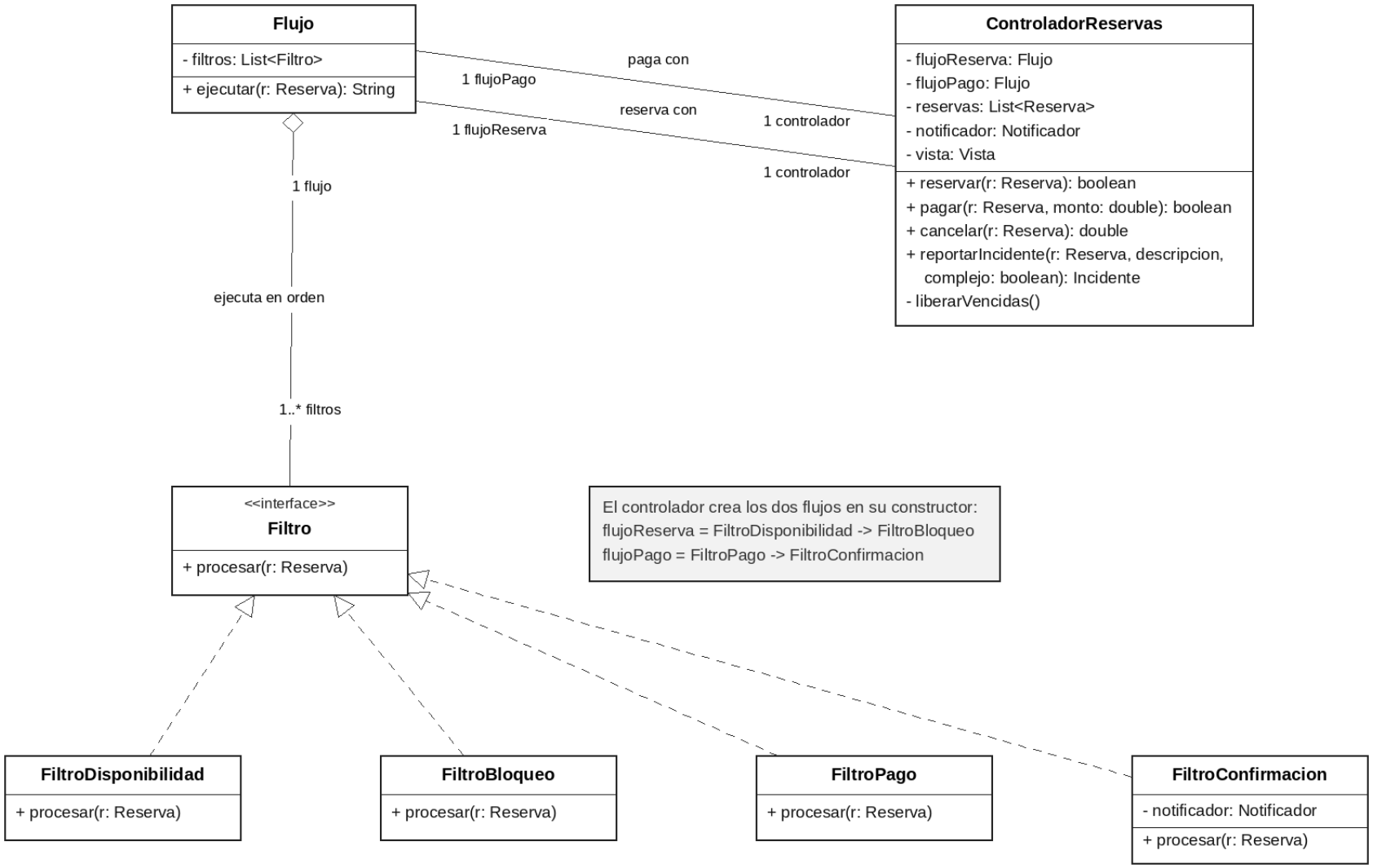



*Figura 4. Diagrama de clases: flujo de datos (controlador, flujo y filtros).*

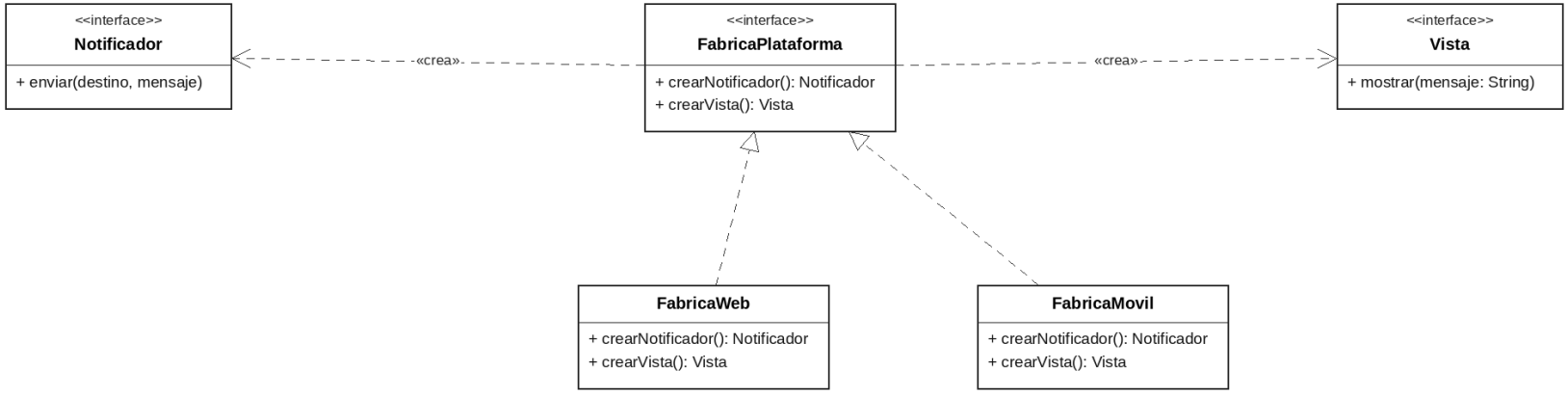



*Figura 5. Diagrama de clases: fábrica abstracta por plataforma (móvil y web).*

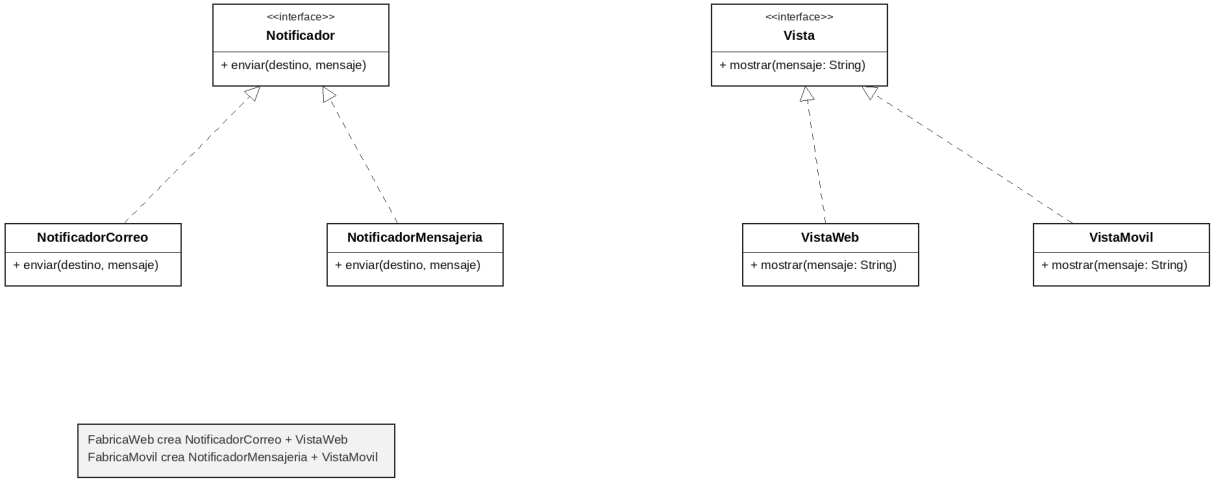



*Figura 6. Diagrama de clases: productos de la fábrica (notificadores y vistas).*

### Justificación de los principios SOLID

| Principio | Aplicación en el diseño |
|---|---|
| **Responsabilidad única** | Cada clase tiene una sola responsabilidad. Reserva guarda los datos y el estado de la reserva; FiltroDisponibilidad, FiltroBloqueo, FiltroPago y FiltroConfirmacion hacen cada uno un solo paso; Flujo solo los ejecuta en orden; ControladorReservas solo coordina; Incidente solo registra el nivel de atención. |
| **Abierto/Cerrado** | Se agregan funciones sin modificar lo existente: un paso nuevo de la reserva es un Filtro más; una plataforma nueva es una fábrica nueva con su Notificador y su Vista; una categoría de servicio nueva es solo otro valor del atributo categoria. |
| **Sustitución de Liskov** | Las implementaciones se usan donde se espera su tipo base sin cambiar el comportamiento: Flujo ejecuta cualquier Filtro; el controlador usa cualquier Notificador y cualquier Vista; Cliente y Administrador pueden usarse como Usuario. |
| **Segregación de interfaces** | Las interfaces son pequeñas: Filtro (procesar), Notificador (enviar), Vista (mostrar) y FabricaPlataforma (crear notificador y vista). Ninguna clase implementa métodos que no usa. |
| **Inversión de dependencias** | ControladorReservas depende de las abstracciones FabricaPlataforma, Notificador y Vista, no de NotificadorCorreo ni VistaMovil; FiltroConfirmacion depende de Notificador y Flujo depende de Filtro. La implementación concreta se elige una sola vez, al crear la fábrica. |

### Aplicación por elemento del diagrama de clases:

| Elemento | Principios | Justificación |
|---|---|---|
| **Usuario, Cliente, Administrador** | S, L | Usuario reúne los datos comunes; Cliente y Administrador agregan solo lo propio de su rol y pueden usarse como Usuario. |
| **Mascota, Cuidador, Centro** | S | Cada clase modela un solo concepto; Centro solo agrupa servicios y cuidadores y permite buscar servicios. |
| **Servicio** | S, O | Guarda los datos y los cupos de un servicio; como la categoría es un atributo, un servicio o categoría nuevos no requieren clases. |
| **Reserva, EstadoServicio, Incidente** | S | Reserva solo guarda los datos y el estado de la reserva (las reglas están en los filtros); Incidente solo registra su nivel de atención. |
| **ControladorReservas** | S, D | Solo coordina los flujos, la lista de reservas, el notificador y la vista; obtiene los dos últimos de la fábrica. |
| **Flujo, Filtro y sus 4 implementaciones** | S, O, L, I | Flujo solo ejecuta una lista de filtros; cada filtro hace un solo paso, todos cumplen el mismo contrato (interfaz de un método) y un paso nuevo es un filtro nuevo. |
| **FabricaPlataforma, FabricaWeb, FabricaMovil** | O, L, D | El controlador depende de la interfaz; cada fábrica crea la familia de su plataforma (notificador y vista). |
| **Notificador y Vista, con sus implementaciones (correo, mensajería, web, móvil)** | O, L, I, D | Interfaces de un solo método: un canal o una vista nuevos son una clase nueva, son intercambiables y separan la presentación de la lógica. |

## Sección C: Diagramas de secuencia

Cada diagrama representa uno de los cuatro casos de uso descritos en la sección A. Los objetos son clases del diagrama de clases (sección B) y los mensajes son sus métodos; «create» indica la creación de un objeto.

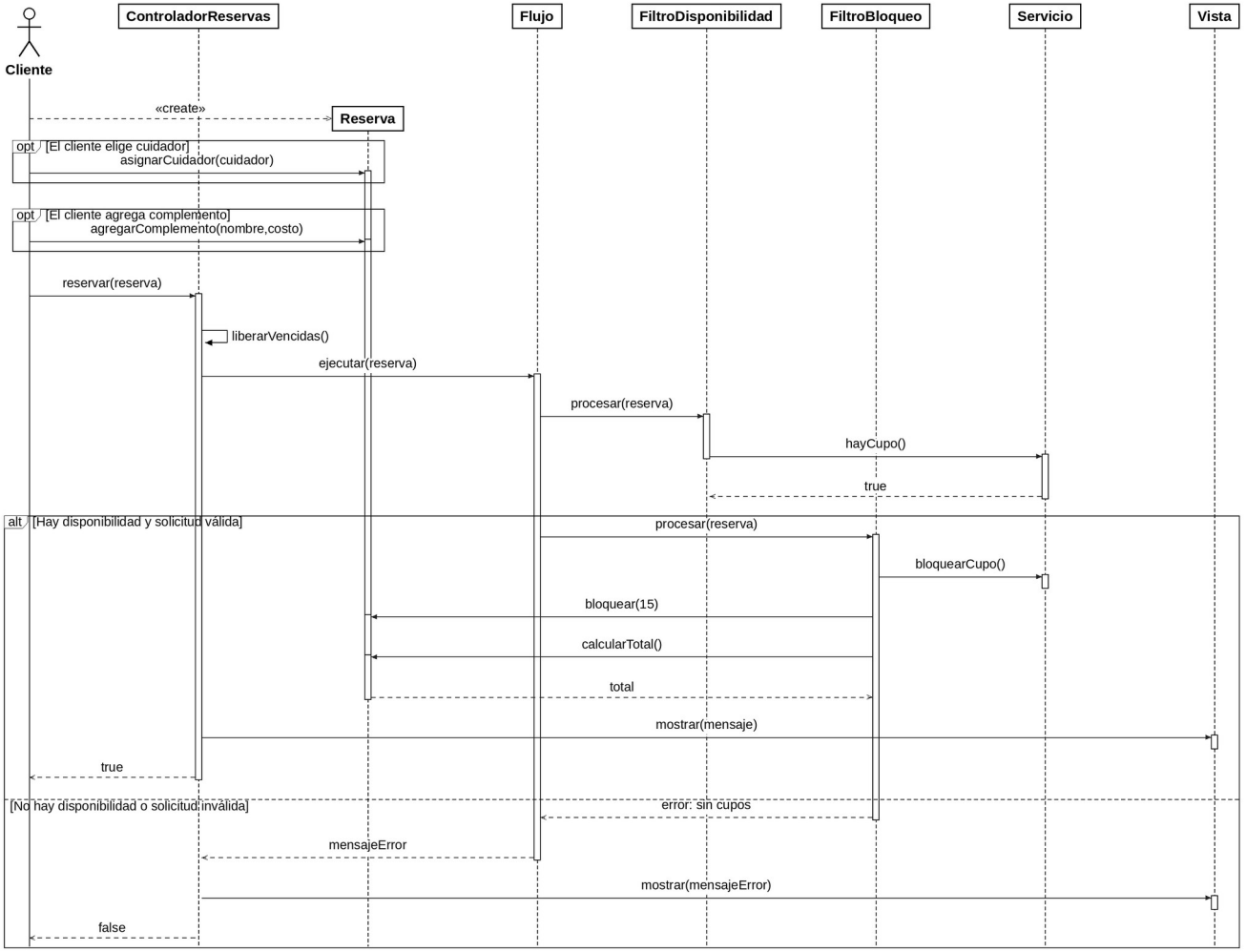



*Figura 7. Diagrama de secuencia del caso de uso 1: Reservar servicio.*

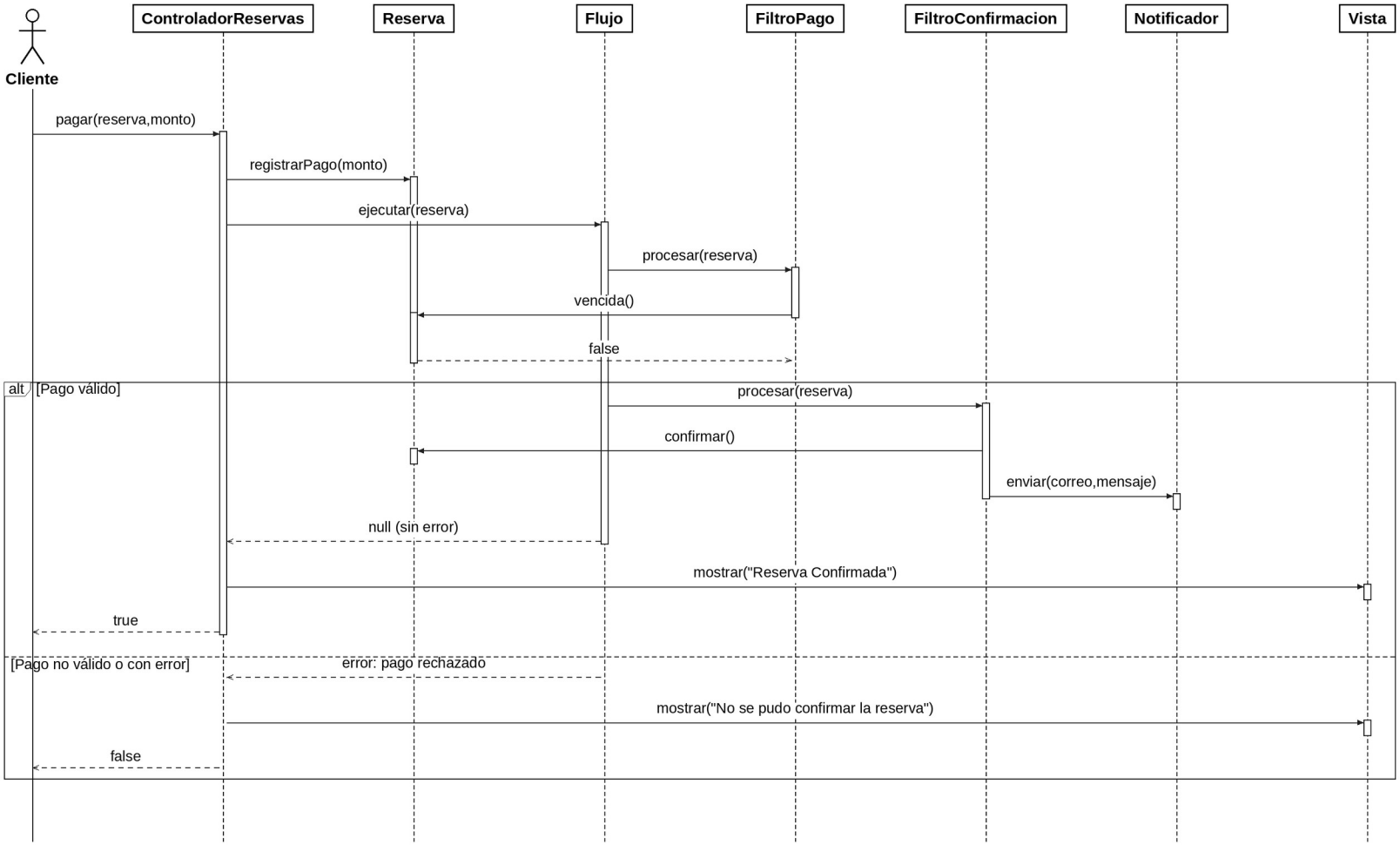



*Figura 8. Diagrama de secuencia del caso de uso 2: Pagar y confirmar reserva.*

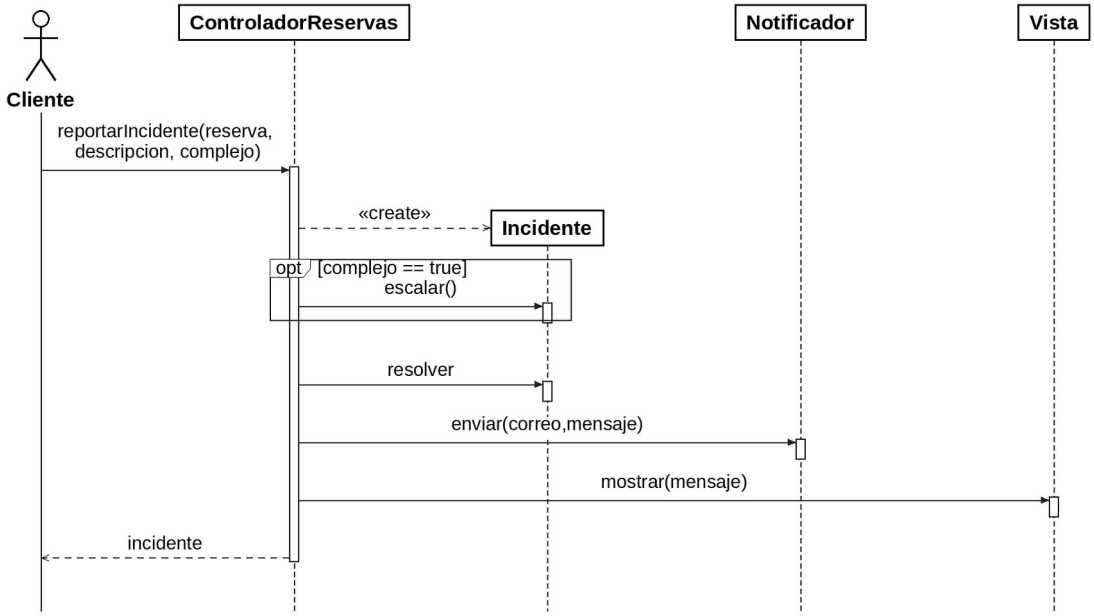



*Figura 9. Diagrama de secuencia del caso de uso 3: Reportar incidente.*

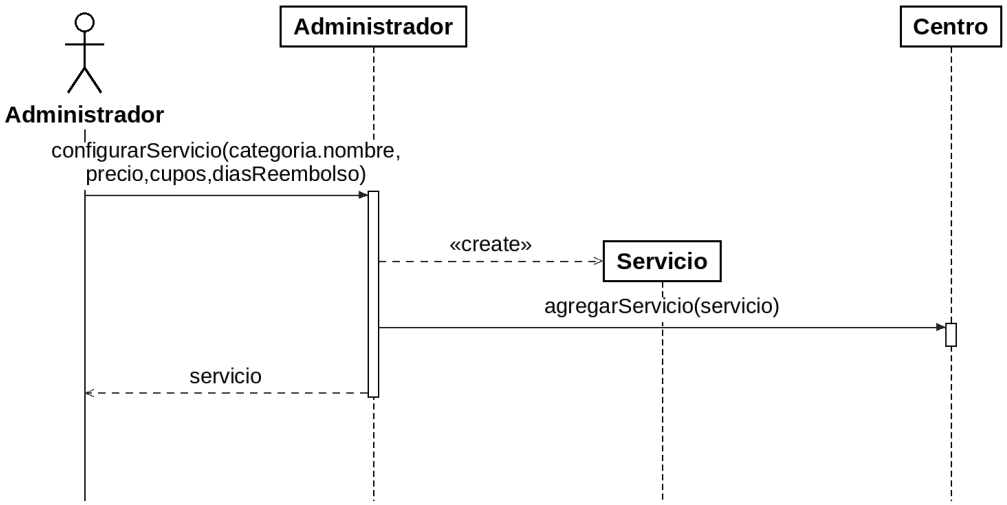



*Figura 10. Diagrama de secuencia del caso de uso 4: Configurar servicio.*

## Sección D: Código base en Java

El código reproduce las clases e interfaces del diagrama de clases (26 en total) con la lógica mínima de cada método. La clase Main simula un recorrido completo (configurar servicios, registrar mascota, reservar, pagar, reportar incidentes y cancelar) en las plataformas móvil y web.

Repositorio de GitHub: https://github.com/JamesSantana911/petdaycare-grupo4